

SIMATS ENGINEERING

ASSESSMENT TOOL

COURSE NAME: Theory of Computation

COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO3: Construct regular expressions, and use the pumping lemma and closure properties to analyze regular languages. (BL:3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total

Marks:100

Duration : 1 Hour

Experiments

Code of Conduct:

I Yuvann Nithish R (192411267) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Yuvann Nithish R

Sl. No. Lab Experiments - Questions

- 1 Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)
$$\rightarrow 0A1 \quad A \rightarrow 0A \mid 1A \mid \epsilon$$
- 2 Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)
$$\rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \epsilon$$
- 3 Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)
$$\rightarrow 0S0 \mid A \quad A \rightarrow 1A \mid \epsilon$$
- 4 Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)
$$\rightarrow 0S1 \mid \epsilon$$
- 5 Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)
$$\rightarrow A101A \quad A \rightarrow 0A \mid 1A \mid \epsilon$$

SIMATS ENGINEERING

ASSESSMENT TOOL

SOLUTIONS

1. CFG: $S \rightarrow 0A1$, $A \rightarrow 0A \mid 1A \mid \varepsilon$

Language

This grammar generates strings that:

Start with 0

End with 1

Have any combination of 0 and 1 in between

Examples: 01, 001, 011, 0101, 001101

C Program

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int n, i, valid = 1;

    printf("Enter a string: ");
    scanf("%s", str);

    n = strlen(str);

    if (n < 2 || str[0] != '0' || str[n - 1] != '1') {
        valid = 0;
    }
}
```

```

for (i = 0; i < n; i++) {
    if (str[i] != '0' && str[i] != '1') {
        valid = 0;
        break;
    }
}

if (valid)
    printf("String belongs to the language.\n");
else
    printf("String does not belong to the language.\n");

return 0;
}

```

Example:

Enter a string: 00101
String belongs to the language.

Output:

The screenshot shows the OneCompiler IDE interface. The editor displays a C program that checks if a string consists only of '0's and '1's. The code is as follows:

```

1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char str[100];
6      int n, i, valid = 1;
7
8      printf("Enter a string: ");
9      scanf("%s", str);
10
11     n = strlen(str);
12
13     if (n < 2 || str[0] != '0' || str[n - 1] != '1') {
14         valid = 0;
15     }
16
17     for (i = 0; i < n; i++) {
18         if (str[i] != '0' && str[i] != '1') {
19             valid = 0;
20             break;
21         }
22     }
23
24     if (valid)
25         printf("String belongs to the language.\n");
26     else
27         printf("String does not belong to the language.\n");
28
29     return 0;
30 }

```

The console output on the right shows:

```

Enter a string: 01
String belongs to the language.

```

2. CFG: $S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \varepsilon$

Language

This grammar generates **palindromes** containing only 0 and 1.

Examples:

ε
0
1
00
11
010
101
0110
1001

C Program

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int i, n, valid = 1;

    printf("Enter a string: ");
    scanf("%s", str);

    n = strlen(str);

    for (i = 0; i < n; i++) {
        if (str[i] != '0' && str[i] != '1') {
            valid = 0;
            break;
        }
    }

    if (valid) {
```

```

        for (i = 0; i < n / 2; i++) {
            if (str[i] != str[n - i - 1]) {
                valid = 0;
                break;
            }
        }
    }

    if (valid)
        printf("String belongs to the language.\n");
    else
        printf("String does not belong to the language.\n");

    return 0;
}

```

Example:

Enter a string: 0110
String belongs to the language.

Because:

0110
↑↑
Palindrome

output :

The screenshot shows the OneCompiler IDE interface. The main editor displays a C program that checks if a string is a palindrome. The code includes headers for `stdio.h` and `string.h`, declares a character array `str` of size 100, and reads a string from the user. It then uses a nested loop structure to compare characters from both ends of the string towards the center. If any pair of characters does not match, it sets `valid` to 0 and breaks the loop. Finally, it prints a message indicating whether the string belongs to the language of palindromes.

```

1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char str[100];
6      int i, n, valid = 1;
7
8      printf("Enter a string: ");
9      scanf("%s", str);
10
11     n = strlen(str);
12
13     for (i = 0; i < n; i++) {
14         if (str[i] != '0' && str[i] != '1') {
15             valid = 0;
16             break;
17         }
18     }
19
20     if (valid) {
21         for (i = 0; i < n / 2; i++) {
22             if (str[i] != str[n - i - 1]) {
23                 valid = 0;
24                 break;
25             }
26         }
27     }
28
29     if (valid)
30         printf("String belongs to the language.\n");
31     else
32         printf("String does not belong to the language.\n");
33
34     return 0;
35 }

```

The right-hand pane shows the console output:

```

Enter a string: 0110
String belongs to the language.

```

The bottom status bar indicates the application is "Ready" and the terminal is active.

3. CFG: $S \rightarrow 0S0 \mid A, A \rightarrow 1A \mid \epsilon$

Language

This generates strings of the form:

$0^n 1^m 0^n$

That means:

Same number of 0s on both sides

Any number of 1s in the middle

Examples:

ϵ
1
11
010
0110
00100
0011100

C Program

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int n, left = 0, right, i;
    int valid = 1;

    printf("Enter a string: ");
    scanf("%s", str);

    n = strlen(str);
    right = n - 1;
```

```

while (left < n && str[left] == '0')
    left++;

while (right >= 0 && str[right] == '0')
    right--;

/* Check that the middle part contains only 1s */
for (i = left; i <= right; i++) {
    if (str[i] != '1') {
        valid = 0;
        break;
    }
}

/* Count zeros on both sides */
if (valid) {
    int leftZeros = left;
    int rightZeros = n - 1 - right;

    if (leftZeros != rightZeros)
        valid = 0;
}

if (valid)
    printf("String belongs to the language.\n");
else
    printf("String does not belong to the language.\n");

return 0;
}

```

Example:

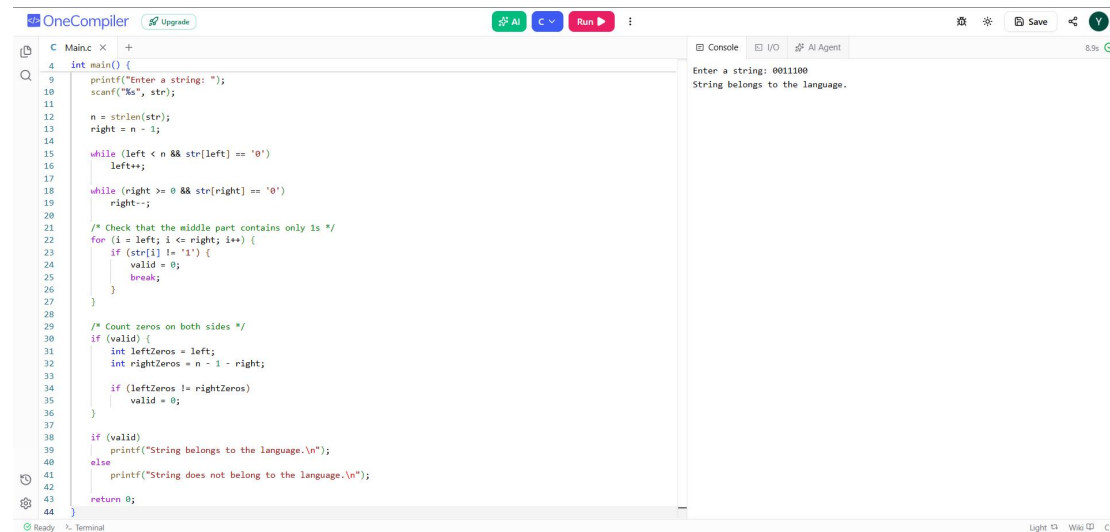
Enter a string: 0011100
String belongs to the language.

Because:

00 111 00

There are **2 zeros on both sides.**

Output:



```
1  int main() {
2      printf("Enter a string: ");
3      scanf("%s", str);
4
5      n = strlen(str);
6      right = n - 1;
7
8      while (left < n && str[left] == '0')
9          left++;
10
11     while (right >= 0 && str[right] == '0')
12         right--;
13
14     /* Check that the middle part contains only 1s */
15     for (i = left; i <= right; i++) {
16         if (str[i] != '1') {
17             valid = 0;
18             break;
19         }
20     }
21
22     /* Count zeros on both sides */
23     if (valid) {
24         int leftZeros = left;
25         int rightZeros = n - 1 - right;
26
27         if (leftZeros != rightZeros)
28             valid = 0;
29     }
30
31     if (valid)
32         printf("String belongs to the language.\n");
33     else
34         printf("String does not belong to the language.\n");
35
36     return 0;
37 }
```

Enter a string: 0011100
String belongs to the language.

4. CFG: $S \rightarrow 0S1 \mid \epsilon$

Language

This grammar generates:

0^n1^n

So the number of 0s must be exactly equal to the number of 1s, with all 0s before all 1s.

Examples:

ϵ

01

0011

000111

00001111

Invalid:

0
1
011
001
0101

C Program

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int n, i, zeros = 0, ones = 0;
    int foundOne = 0;
    int valid = 1;

    printf("Enter a string: ");
    scanf("%s", str);

    n = strlen(str);

    for (i = 0; i < n; i++) {

        if (str[i] == '0') {
            if (foundOne) {
                valid = 0;
                break;
            }
            zeros++;
        }
        else if (str[i] == '1') {
            foundOne = 1;
            ones++;
        }
        else {
            valid = 0;
            break;
        }
    }
}
```

```

    if (zeros != ones)
        valid = 0;

    if (valid)
        printf("String belongs to the language.\n");
    else
        printf("String does not belong to the language.\n");

    return 0;
}

```

Example:

Enter a string: 000111
String belongs to the language.

Because:

000 111
3 3

output:

The screenshot shows the OneCompiler IDE interface. The main editor displays a C program that checks if a string belongs to a language based on the count of '0's and '1's. The code is as follows:

```

1  int main() {
2      int valid = 1;
3
4      printf("Enter a string: ");
5      scanf("%s", str);
6
7      n = strlen(str);
8
9      for (i = 0; i < n; i++) {
10         if (str[i] == '0') {
11             if (foundOne) {
12                 valid = 0;
13                 break;
14             }
15             zeros++;
16         }
17         else if (str[i] == '1') {
18             foundOne = 1;
19             ones++;
20         }
21         else {
22             valid = 0;
23             break;
24         }
25     }
26
27     if (zeros != ones)
28         valid = 0;
29
30     if (valid)
31         printf("String belongs to the language.\n");
32     else
33         printf("String does not belong to the language.\n");
34
35     return 0;
36 }

```

The console output on the right shows the program's execution:

```

Enter a string: 000111
String belongs to the language.

```

The status bar at the bottom indicates the program is ready and running in a terminal window.

5. CFG: $S \rightarrow A101A$, $A \rightarrow 0A \mid 1A \mid \epsilon$

Language

Since A can generate **any binary string**, the grammar generates:

(binary string) 101 (binary string)

In simple terms:

The string must contain 101 as a substring.

Examples:

101
0101
1010
11010
001011

C Program

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int n, i, found = 0;

    printf("Enter a string: ");
    scanf("%s", str);

    n = strlen(str);

    /* Check whether all characters are 0 or 1 */
    for (i = 0; i < n; i++) {
        if (str[i] != '0' && str[i] != '1') {
            printf("String does not belong to the language.\n");
            return 0;
        }
    }

    /* Search for substring 101 */
```

```

for (i = 0; i <= n - 3; i++) {
    if (str[i] == '1' &&
        str[i + 1] == '0' &&
        str[i + 2] == '1') {

        found = 1;
        break;
    }
}

if (found)
    printf("String belongs to the language.\n");
else
    printf("String does not belong to the language.\n");

return 0;
}

```

Example:

Enter a string: 110101
String belongs to the language.

Because it contains:

11 101 01
↑↑↑

output:

The screenshot shows the OneCompiler IDE interface. The code editor on the left contains the following C code:

```

1  #include <string.h>
2
3  int main() {
4      char str[100];
5      int n, i, found = 0;
6
7      printf("Enter a string: ");
8      scanf("%s", str);
9
10     n = strlen(str);
11
12     /* Check whether all characters are 0 or 1 */
13     for (i = 0; i < n; i++) {
14         if (str[i] != '0' && str[i] != '1') {
15             printf("String does not belong to the language.\n");
16             return 0;
17         }
18     }
19
20     /* Search for substring 101 */
21     for (i = 0; i <= n - 3; i++) {
22         if (str[i] == '1' &&
23             str[i + 1] == '0' &&
24             str[i + 2] == '1') {
25             found = 1;
26             break;
27         }
28     }
29
30     if (found)
31         printf("String belongs to the language.\n");
32     else
33         printf("String does not belong to the language.\n");
34
35     return 0;
36 }

```

The right-hand side of the IDE shows the output console with the following text:

```

Enter a string: 110101
String belongs to the language.

```

The status bar at the bottom indicates the compiler is ready and the terminal is active.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand